
作业报告

课程名称： 算法设计与分析

作业题目：

专业：

班级：

学号：

姓名：

联系方式：

邮箱：

2025 年 12 月 16 日

1.概述思路

(1) 问题背景与目标说明

在日常生活中，排队接水是一种常见的场景。抽象到算法问题中，假设有 n 个人在同一个水龙头前排队接水，每个人接水所需的时间为 T_i ，且所有 T_i 互不相同。

定义**等待时间**为：某个人在开始接水前，需要等待其前面所有人接水所花费的时间总和。

本题的目标是：**通过合理安排排队顺序，使所有人的平均等待时间最小。**

(2) 贪心算法的应用

贪心选择策略：为了使平均等待时间最小，应尽量减少后面每个人所需要等待的时间。直观地看，如果某个接水时间较长的人排在前面，会显著增加其后所有人的等待时间。因此可以得出结论：**接水时间越短的人，应越早接水。**据此，本题采用的贪心策略为：**每次优先选择接水时间最短的人排在当前最前面。**

(3) 贪心算法正确性分析

贪心选择性质：在当前尚未安排顺序的人员中，若选择接水时间最短的人排在最前面，可以使其接水时间对后续所有人的影响最小。如果选择接水时间较长的人提前接水，则会无谓地增加后续所有人的等待时间总和，因此该贪心选择是合理且必要的。

最优子结构性质：假设已经确定了前 $k-1$ 个人的最优排队顺序，那么剩余人员仍然构成一个规模更小、结构相同的问题。此时，第 k 个位置依然应选择剩余人员中接水时间最短的人。因此，该问题满足最优子结构性质，贪心算法在本题中是正确且可行的。

(4) 算法步骤说明

排序阶段：将所有人的接水时间 T_i 按从小到大排序。

等待时间计算阶段：第1个人的等待时间为0；第二个人的等待时间为第一个人的 T_1 。第三个人的等待时间为前两个人的 T_1 之和。

结果计算阶段：将所有人的等待时间累加后，除以人数 n ，得到平均等待时间，并按要求保留一位小数。

(5) 复杂度分析

在本算法中，主要包含两个核心操作：**排序操作**和**等待时间的计算操作**。

首先，在**排序阶段**，需要对 n 个接水时间 T_i 进行从小到大的排序。由于采用的是基于比较的排序算法（如快速排序或归并排序），其**平均时间复杂度为 $O(n \log n)$** 。该步骤是整个算法中最耗时的部分。

其次，在**等待时间计算阶段**，算法只需按照排序后的顺序对数组进行一次线性遍历。通过维护一个累计接水时间的变量，将其不断累加到总等待时间中。该过程不涉及嵌套循环，因此**时间复杂度为 $O(n)$** 。

由于排序阶段的时间复杂度高于等待时间计算阶段，整体算法的时间复杂度由排序操作主导，**最终时间复杂度为 $O(n \log n)$** 。

从空间复杂度角度来看，算法仅使用了一个长度为 n 的数组来存储接水时间，以及若干辅助变量用于累计计算，未使用额外的复杂数据结构，因此**空间复杂度为 $O(n)$** 。

综上所述，该算法在时间和空间效率上均较为合理，能够在保证结果最优的前提下，高效地解决排队接水的平均等待时间最小化问题。

2.代码

```
#include <bits/stdc++.h>
#include <iomanip>          // 用于 fixed / setprecision 设置小数输出格式
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(NULL);

    int n;                // n 表示排队人数
    cin >> n;             // 读入人数

    vector<int> Ti(n);     // Ti[i] 表示第 i 个人接水需要的时间
    for (int i = 0; i < n; ++i) {
        cin >> Ti[i];     // 读入每个人的接水时间
    }

    // 贪心策略: 为了让平均等待时间最小, 应该让接水时间短的人先接水
    // 因此将所有接水时间从小到大排序, 得到最优排队顺序
    sort(Ti.begin(), Ti.end());

    long long sum_wait = 0;
    // sum_wait: 所有人的等待时间之和 (用 long long 防止溢出)
    long long current_sum = 0;
    // current_sum: 当前已经接完水的总用时 (即“前面所有人的接水时间之和”)

    // 按照排好的顺序依次计算每个人的等待时间
    for (int i = 0; i < n; ++i) {
        // 第 i 个人的等待时间 = 他前面所有人的接水时间总和 = current_sum
        sum_wait += current_sum;

        // 当前这个人接水结束后, 总用时要加上他的接水时间
        // 这样下一位人的等待时间就会在这个基础上继续累加
        current_sum += Ti[i];
    }

    // 平均等待时间 = 总等待时间 / 人数
    // sum_wait 是 long long, 为了得到小数结果需要转成 double
    double average_wait = (double)sum_wait / n;

    // 按题目要求: 输出保留 1 位小数
    cout << fixed << setprecision(1) << average_wait;

    return 0;
}
```

3.测试结果

样例测试：

输入如下：

4 2 3 4 5

运行结果：

```
F:\Upan\课件\大三上\算法设计 × + - □ ×
4 2 3 4 5
4.0
-----
Process exited after 17.62 seconds with return value 0
请按任意键继续. . .
```

解释：排序后的接水时间为 2, 3, 4, 5

等待时间计算：

第 1 个人：0

第 2 个人：2

第 3 个人：2 + 3 = 5

第 4 个人：2 + 3 + 4 = 9

总等待时间：0 + 2 + 5 + 9 = 16

平均等待时间：16 / 4 = 4.0

测试用例 1：

6 4 3 2 1 6 5

运行结果：

```
F:\Upan\课件\大三上\算法设计 × + - □ ×
6 4 3 2 1 6 5
5.8
-----
Process exited after 4.438 seconds with return value 0
请按任意键继续. . .
```

解释：排序后的接水时间为 1, 2, 3, 4, 5, 6

等待时间计算：

第 1 个人：0

第 2 个人：1

第 3 个人：1 + 2 = 3

第 4 个人：1 + 2 + 3 = 6

第 4 个人：1 + 2 + 3 + 4 = 10

第 4 个人：1 + 2 + 3 + 4 + 5 = 15

总等待时间：0 + 1 + 3 + 6 + 10 + 15 = 35

平均等待时间：35 / 6 ≈ 5.833 → 5.8

测试用例 2：

8 8 7 1 9 3 6 2 5

运行结果：

```

F:\Upan\课件\大三上\算法设计 × + v
8 8 7 1 9 3 6 2 5
11.8
-----
Process exited after 1.79 seconds with return value 0
请按任意键继续. . .

```

解释：排序后的接水时间为 1, 2, 3, 5, 6, 7, 8, 9

等待时间计算：

第 1 个人：0

第 2 个人：1

第 3 个人：1 + 2 = 3

第 4 个人：1 + 2 + 3 = 6

第 5 个人：1 + 2 + 3 + 5 = 11

第 6 个人：1 + 2 + 3 + 5 + 6 = 17

第 7 个人：1 + 2 + 3 + 5 + 6 + 7 = 24

第 8 个人：1 + 2 + 3 + 5 + 6 + 7 + 8 = 32

总等待时间：0 + 1 + 3 + 6 + 11 + 17 + 24 + 32 = 94

平均等待时间：94 / 8 = 11.75 → 11.8

测试用例 3：

10 10 9 8 7 6 5 4 3 2 1

运行结果：

```

F:\Upan\课件\大三上\算法设计 × + v - □ ×
10 10 9 8 7 6 5 4 3 2 1
16.5
-----
Process exited after 1.323 seconds with return value 0
请按任意键继续. . .

```

解释：排序后的接水时间为 1, 2, 3, 4, 5, 6, 7, 8, 9, 10

等待时间计算：

第 1 个人：0

第 2 个人：1

第 3 个人：1 + 2 = 3

第 4 个人：1 + 2 + 3 = 6

第 5 个人：1 + 2 + 3 + 4 = 10

第 6 个人：1 + 2 + 3 + 4 + 5 = 15

第 7 个人：1 + 2 + 3 + 4 + 5 + 6 = 21

第 8 个人：1 + 2 + 3 + 4 + 5 + 6 + 7 = 28

第 9 个人：1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 = 36

第 10 个人：1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9 = 45

总等待时间：0 + 1 + 3 + 6 + 10 + 15 + 21 + 28 + 36 + 45 = 165

平均等待时间：165 / 10 = 16.5

测试用例 4：

15 10 20 30 40 50 60 70 80 90 100 110 120 130 140 150

运行结果：

```

F:\Upan\课件\大三上\算法设计  ×  +  ∨  -  □  ×
15 10 20 30 40 50 60 70 80 90 100 110 120 130 140 150
373.3
-----
Process exited after 1.353 seconds with return value 0
请按任意键继续. . .
    
```

解释：排序后的接水时间为 10, 20, 30, 40, 50, 60, 70, 80, 90, 100, 110, 120, 130, 140, 150
等待时间计算：

第 1 个人：0

第 2 个人：10

第 3 个人：10 + 20 = 30

第 4 个人：10 + 20 + 30 = 60

第 5 个人：10 + 20 + 30 + 40 = 100

第 6 个人：10 + 20 + 30 + 40 + 50 = 150

第 7 个人：10 + 20 + 30 + 40 + 50 + 60 = 210

第 8 个人：10 + 20 + 30 + 40 + 50 + 60 + 70 = 280

第 9 个人：10 + 20 + 30 + 40 + 50 + 60 + 70 + 80 = 360

第 10 个人：10 + 20 + 30 + 40 + 50 + 60 + 70 + 80 + 90 = 450

第 11 个人：10 + 20 + 30 + 40 + 50 + 60 + 70 + 80 + 90 + 100 = 550

第 12 个人：10 + 20 + 30 + 40 + 50 + 60 + 70 + 80 + 90 + 100 + 110 = 660

第 13 个人：10 + 20 + 30 + 40 + 50 + 60 + 70 + 80 + 90 + 100 + 110 + 120 = 780

第 14 个人：10 + 20 + 30 + 40 + 50 + 60 + 70 + 80 + 90 + 100 + 110 + 120 + 130 = 910

第 15 个人：10 + 20 + 30 + 40 + 50 + 60 + 70 + 80 + 90 + 100 + 110 + 120 + 130 + 140 = 1050

总等待时间：0 + 10 + 30 + 60 + 100 + 150 + 210 + 280 + 360 + 450 + 550 + 660 + 780 + 910 + 1050 = 5600

平均等待时间：5600 / 15 = 373.333... → 373.3